Network Simulation & Emulation Environment (NSE2) Manual

Lars Baumgaertner

2025-07-02 Work in Progress

Table of Contents

Chapter 1. Introduction

The Network Simulation & Emulation Environment (NSE2) is a powerful tool designed to facilitate the simulation and emulation of network environments. This manual provides a comprehensive guide to installing, configuring, and using NSE2 effectively. It also includes instructions for creating new simulation scenarios and understanding the file formats used by NSE2.

NSE2 is built on top of Docker and Docker Compose, allowing users to create complex network topologies and simulate various network conditions. It is intended for researchers, developers, and network engineers who need to test and validate network protocols, applications, and configurations in a controlled realtime environment.

Chapter 2. Installation

NSE2 was designed to be easy to install and set up, allowing users to quickly get started with network simulation and emulation. While it has been designed mainly for Linux systems, it can in theory run on any system that supports Docker. This section provides detailed instructions on how to install NSE2 on your system.

2.1. Prerequisites

Before installing NSE2, ensure that you have the following prerequisites:

- A compatible operating system (Linux, macOS, or Windows)
- Python 3.10 or higher
- pip (Python package installer)
- Git (for cloning the repository)
- Docker (for containerized environments)
- Docker Compose

In theory, podman can be used instead of Docker, but it is not tested. Also, it must be used with the proper `docker compose` and not `podman-compose`.

2.2. Installation Steps

1. Clone the NSE2 repository from GitHub: `git clone https://github.com/esa/nse2.git`
2. Navigate to the NSE2 directory: `cd nse2`
3. Create a virtual environment: `python -m venv .venv`
4. Activate the virtual environment: `source .venv/bin/activate`
5. Install the required Python packages: `pip install -r requirements.txt`
6. Install NSE2: `./install_symlinks.sh`

Chapter 3. Running NSE2

In this section, we will cover how to run NSE2, including starting a simulation as well as interacting with the simulation using the Command Line Interface (CLI) (Section 3.3), the Management Web UI (Section 3.4) and the visualization frontend (Section 3.5). We will also provide an example scenario to demonstrate how to set up and run a simulation in NSE2.

3.1. Example Scenario Overview

For demonstration purposes, we will use the `simple` scenario, located in `scenarios/simple/`, which is a basic network simulation scenario included with NSE2. Here, we have three nodes (n1, n2, and n3) connected in a straight line (Figure 1).

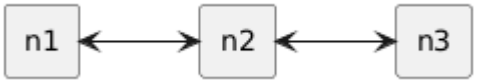

Figure 1. Network topology diagram of the `simple` scenario

In our example scenario we can find the following files:

```
$ ls scenarios/simple/
compose.yml  contacts.ccp  actions.txt viz.json run.sh
```

This is the minimal set of files required to run a simulation in NSE2 that has a topology, fluctuating contacts, and automated actions plus some optional information for the visualization frontend.

The `compose.yml` file defines the network topology and the nodes in the simulation, while the `contacts.ccp` file defines the contact plan for the nodes, the `actions.txt` file defines actions to be performed during the simulation, and the `viz.json` file provides visualization data.

The example `compose.yml` file (Listing 1) defines three nodes (n1, n2, and n3) connected in a straight line. Each node is configured with an IP address and a Docker image to run. The network interfaces are defined in the `networks` section, and the Docker Compose file specifies how the nodes are connected. A detailed description of the compose file format can be found in Section 5.1.

The docker image must contain `tc` and `iproute2` tools to allow for network configuration and traffic control. In our example, we use the Alpine Linux image, which is lightweight and suitable for this purpose but may require additional packages to be installed.

Listing 1. Compose file of the simple scenario.

```
name: simple test scenario
services:
  n1:
    container_name: n1
    hostname: n1
    cap_add:
```

```
    - NET_ADMIN
    privileged: 'true'
    volumes:
    - ./compose.yml:/compose.yml:ro
    environment:
    - NODE_ID=1
    - TYPE=Host
    networks:
      n1_n2:
        ipv4_address: 172.33.0.2
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
  n2:
    container_name: n2
    hostname: n2
    cap_add:
    - NET_ADMIN
    privileged: 'true'
    volumes:
    - ./compose.yml:/compose.yml:ro
    environment:
    - NODE_ID=2
    - TYPE=Host
    networks:
      n1_n2:
        ipv4_address: 172.33.0.3
      n2_n3:
        ipv4_address: 172.33.1.2
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
  n3:
    container_name: n3
    hostname: n3
    cap_add:
    - NET_ADMIN
    privileged: 'true'
    volumes:
    - ./compose.yml:/compose.yml:ro
    environment:
    - NODE_ID=3
    - TYPE=Host
    networks:
      n2_n3:
        ipv4_address: 172.33.1.3
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
networks:
  n1_n2:
    driver: bridge
    ipam:
      config:
```

```
      - subnet: 172.33.0.0/24
    driver_opts:
      com.docker.network.container_iface_prefix: n1_n2_
  n2_n3:
    driver: bridge
    ipam:
      config:
      - subnet: 172.33.1.0/24
    driver_opts:
      com.docker.network.container_iface_prefix: n2_n3_
```

The contact plan (Listing 2) defines the communication links between the nodes, including bandwidth, delay, loss, and jitter. In our example, we have a fixed contact between n1 and n2 with a bandwidth of 100 Mbps and a delay of 100 ms, and a fluctuating contact between n2 and n3 with a bandwidth of 2 Mbps, a delay of 10 ms, and a time window from 20 to 40 seconds. Also, we set the contact plan to loop, meaning that the contacts will be repeated indefinitely. A detailed description of the contacts file format can be found in [contacts-format].

Listing 2. Core contact plan of the simple scenario.

```
s loop 1
a fixed n1 n2 100000000 0.0 0.1 0.0
a fixed n2 n1 100000000 0.0 0.1 0.0
a contact 20 40 n2 n3 2000000 0.0 0.01 0.0
a contact 20 40 n3 n2 2000000 0.0 0.01 0.0
```

The actions file (Listing 3) defines the actions to be performed during the simulation. In our example, at first we log the start time on each node, then we schedule a repeated process on each node to log the link properties every 10 seconds by sending ping requests between the nodes. The actions file also includes a cleanup step that logs the finish time and terminates long-running processes at the end of the simulation. A detailed description of the actions file format can be found in Section 5.3.

Listing 3. Example actions file of the simple scenario.

```
# DURATION is optional
# DURATION=300

# start a process to log the link properties every 10 seconds on the ground stations
EXEC 0 n1 "date > /tmp/started.txt"
EXEC 0 n2 "date > /tmp/started.txt"
EXEC 0 n3 "date > /tmp/started.txt"

# at timestamp 5 start to send ping (200 bytes) every 10s to gs1 from eosat
REPEAT 5 10 n1 "(date ; ping -c 1 -s 200 -W1 -w1 n2; echo "----------------------") >>
/tmp/n2.txt"

# at timestamp 10 start to send ping (200 bytes) every 10s to gs2 from eosat
REPEAT 10 10 n2 "(date ; ping -c 1 -s 200 -W1 -w1 n3; echo "----------------------")
>> /tmp/n3.txt"

# be nice and shut down long running processes
FINALLY n1 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n2 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n3 "date > /tmp/finished.txt && pkill -f ping"
```

3.2. Starting a Simulation

The following steps need to be performed to start a simulation in NSE2:

1. Start the topology: `nse2_topo compose.yml`
2. Start the contacts: `nse2_contacts -m -s contacts.ccp`
3. Start the actions: `nse2_actions actions.txt`
4. Optionally, start the node manager: `nse2_mgr compose.yml contacts.ccp`

HINT: Use a terminal multiplexer like `tmux` or `screen` to run these commands in separate windows, allowing you to monitor the simulation in real-time.

Here, an example session starting the simulation.

```
$ nse2_topo compose.yml
Running scenario compose.yml
[+] Running 5/5
 ⠿ Network simpletestscenario_n1_n2  Created                                    0.0s
 ⠿ Network simpletestscenario_n2_n3  Created                                    0.0s
 ⠿ Container n2                      Started                                    0.4s
 ⠿ Container n3                      Started                                    0.4s
 ⠿ Container n1                      Started                                    0.4s
n1  | fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
n2  | fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
n3  | fetch https://dl-cdn.alpinelinux.org/alpine/v3.21/main/x86_64/APKINDEX.tar.gz
```

```
n2  | fetch https://dl-
cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
n1  | fetch https://dl-
cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
n3  | fetch https://dl-
cdn.alpinelinux.org/alpine/v3.21/community/x86_64/APKINDEX.tar.gz
n2  | (1/13) Installing ncurses-terminfo-base (6.5_p20241006-r3)
n2  | (2/13) Installing libncursesw (6.5_p20241006-r3)
n2  | (3/13) Installing readline (8.2.13-r0)
n2  | (4/13) Installing bash (5.2.37-r0)
n2  | Executing bash-5.2.37-r0.post-install
n2  | (5/13) Installing libcap2 (2.71-r0)
n2  | (6/13) Installing zstd-libs (1.5.6-r2)
n2  | (7/13) Installing libelf (0.191-r0)
n2  | (8/13) Installing libmnl (1.0.5-r2)
n2  | (9/13) Installing iproute2-minimal (6.11.0-r0)
n1  | (1/13) Installing ncurses-terminfo-base (6.5_p20241006-r3)
n2  | (10/13) Installing libxtables (1.8.11-r1)
n3  | (1/13) Installing ncurses-terminfo-base (6.5_p20241006-r3)
n2  | (11/13) Installing iproute2-tc (6.11.0-r0)
n1  | (2/13) Installing libncursesw (6.5_p20241006-r3)
n3  | (2/13) Installing libncursesw (6.5_p20241006-r3)
n2  | (12/13) Installing iproute2-ss (6.11.0-r0)
n1  | (3/13) Installing readline (8.2.13-r0)
n2  | (13/13) Installing iproute2 (6.11.0-r0)
n3  | (3/13) Installing readline (8.2.13-r0)
n2  | Executing iproute2-6.11.0-r0.post-install
n2  | Executing busybox-1.37.0-r12.trigger
n2  | OK: 11 MiB in 28 packages
n1  | (4/13) Installing bash (5.2.37-r0)
n3  | (4/13) Installing bash (5.2.37-r0)
n1  | Executing bash-5.2.37-r0.post-install
n1  | (5/13) Installing libcap2 (2.71-r0)
n3  | Executing bash-5.2.37-r0.post-install
n3  | (5/13) Installing libcap2 (2.71-r0)
n1  | (6/13) Installing zstd-libs (1.5.6-r2)
n3  | (6/13) Installing zstd-libs (1.5.6-r2)
n1  | (7/13) Installing libelf (0.191-r0)
n3  | (7/13) Installing libelf (0.191-r0)
n1  | (8/13) Installing libmnl (1.0.5-r2)
n3  | (8/13) Installing libmnl (1.0.5-r2)
n1  | (9/13) Installing iproute2-minimal (6.11.0-r0)
n3  | (9/13) Installing iproute2-minimal (6.11.0-r0)
n1  | (10/13) Installing libxtables (1.8.11-r1)
n3  | (10/13) Installing libxtables (1.8.11-r1)
n1  | (11/13) Installing iproute2-tc (6.11.0-r0)
n3  | (11/13) Installing iproute2-tc (6.11.0-r0)
n1  | (12/13) Installing iproute2-ss (6.11.0-r0)
n3  | (12/13) Installing iproute2-ss (6.11.0-r0)
n1  | (13/13) Installing iproute2 (6.11.0-r0)
n3  | (13/13) Installing iproute2 (6.11.0-r0)
```

```
n1  | Executing iproute2-6.11.0-r0.post-install
n1  | Executing busybox-1.37.0-r12.trigger
n1  | OK: 11 MiB in 28 packages
n3  | Executing iproute2-6.11.0-r0.post-install
n3  | Executing busybox-1.37.0-r12.trigger
n3  | OK: 11 MiB in 28 packages
```

Now all three nodes are running and ready to be configured.

Next step is to start the contact player, which will establish the communication links between the nodes based on the contact plan defined in `contacts.ccp`.

```
$ nse2_contacts -m contacts.ccp
Loading scenario from compose.yml.
Node ipn:1.0 connected to network n1_n2 with 172.33.0.2
Node ipn:2.0 connected to network n1_n2 with 172.33.0.3
Node ipn:2.0 connected to network n2_n3 with 172.33.1.2
Node ipn:3.0 connected to network n2_n3 with 172.33.1.3
Created 3 nodes.
[('n1', 'n2', '-'), ('n2', 'n3', '-')]
['n1', 'n2', '100000000', '0.0', '0.1', '0.0'] 6
CoreContact(timespan=(0, 0), nodes=('n1', 'n2'), bw=100000000, loss=0.000000,
delay=0.100000, jitter=0.000000)
['n2', 'n1', '100000000', '0.0', '0.1', '0.0'] 6
CoreContact(timespan=(0, 0), nodes=('n2', 'n1'), bw=100000000, loss=0.000000,
delay=0.100000, jitter=0.000000)
['20', '40', 'n2', 'n3', '2000000', '0.0', '0.01', '0.0'] 8
CoreContact(timespan=(20, 40), nodes=('n2', 'n3'), bw=2000000, loss=0.000000,
delay=0.010000, jitter=0.000000)
['20', '40', 'n3', 'n2', '2000000', '0.0', '0.01', '0.0'] 8
CoreContact(timespan=(20, 40), nodes=('n3', 'n2'), bw=2000000, loss=0.000000,
delay=0.010000, jitter=0.000000)
all contacts sorted pairs:  [('n2', 'n3', '-'), ('n2', 'n3', '-')]
links:  [('n1', 'n2', '-')]
Link:  ('n1', 'n2', '-')
new link:  ('n1', 'n2', '-')
Updating network map tmp/compose.netmap
Setting up tc for n3 on device n2_n3_0 with 100% loss
0ms
Setting up tc for n2 on device n2_n3_0 with 100% loss
0ms
Activating fixed contact CoreContact(timespan=(0, 0), nodes=('n1', 'n2'),
bw=100000000, loss=0.000000, delay=0.100000, jitter=0.000000)
0.1ms
Activating fixed contact CoreContact(timespan=(0, 0), nodes=('n2', 'n1'),
bw=100000000, loss=0.000000, delay=0.100000, jitter=0.000000)
0.1ms
[ 0 ] Next event(s) at 20
[ 20 ] Activating CoreContact(timespan=(20, 40), nodes=('n2', 'n3'), bw=2000000,
loss=0.000000, delay=0.010000, jitter=0.000000)
0.01ms
[ 20 ] Activating CoreContact(timespan=(20, 40), nodes=('n3', 'n2'), bw=2000000,
loss=0.000000, delay=0.010000, jitter=0.000000)
0.01ms
Link:  ('n1', 'n2', '-')
new link:  ('n1', 'n2', '-')
Link:  ('n2', 'n3', '.')
new link:  ('n2', 'n3', '.')
Updating network map tmp/compose.netmap
[ 20 ] Next event(s) at 40
```

Now, we can start the actions defined in `actions.txt`, which will execute the actions during the

simulation.

```
$ nse2_actions actions.txt
Warning: DURATION variable is not, REPEATing till infinity
0 5 10
[0] Executing commands on: n1 n2 n3
[0]   n1: date > /tmp/started.txt
[0]   n2: date > /tmp/started.txt
[0]   n3: date > /tmp/started.txt
[5] Executing commands on: n1
[5] / 10  n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[5]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[10] Executing commands on: n2
[10] / 10  n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[10]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[15]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[20]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[25]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[30]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[35]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[40]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[45]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[50]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[55]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[60]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[65]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[70]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[75]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[80]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[85]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[90]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
```

```
/tmp/n3.txt
[95]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
[100]   n2: (date ; ping -c 1 -s 200 -W1 -w1 n3; echo ----------------------) >>
/tmp/n3.txt
[105]   n1: (date ; ping -c 1 -s 200 -W1 -w1 n2; echo ----------------------) >>
/tmp/n2.txt
^CCleaning up...
   n3: date > /tmp/finished.txt && pkill -f ping ;  true
   n2: date > /tmp/finished.txt && pkill -f ping ;  true
   n1: date > /tmp/finished.txt && pkill -f ping ;  true
```

3.3. Interacting with the Simulation using the Command Line Interface (CLI)

Once the simulation is running, you can interact with it using the NSE2 Command Line Interface (CLI). With `nse2_sh` you can connect to any running node and start using it interactively.

```
$ nse2_sh n1
n1:/# ls /tmp/
n2.txt started.txt
n1:/# ping n2
PING n2 (172.33.0.3): 56 data bytes
64 bytes from 172.33.0.3: seq=0 ttl=64 time=0.271 ms
64 bytes from 172.33.0.3: seq=1 ttl=64 time=0.329 ms
^C
--- n2 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.271/0.300/0.329 ms
```

You can use all standard docker tools to interact with the nodes, such as `docker exec`, `docker logs`, and `docker ps`.

You can also interact with the simulation contacts backend using `nse2_cmd`. It can be used to get information about the current scenario, active links, simulation time and control the playback of the contacts file, e.g., pause/resume, skip to next event, etc.

3.4. Interacting with the Simulation using the Management Web UI

To control the simulation, you can use the Management Web UI (`nse2_mgr`), which provides a graphical interface to manage the simulation. Amongst others, it provides you with the following features: - View the current state of the simulation, including the nodes and contacts. - Access to shell and logs of each node. - Pause and skipping ahead for the contacts. - View and modification of dynamically managed contacts. - Activate and deactivate contacts. - Change bandwidth, delay, loss, and jitter of contacts.

Opening the logs or a terminal requires X11 forwarding (set `DISPLAY` environment variable) and an installed `xterm` on the host system running the manager.

3.5. Visualizing the Simulation

To visualize the simulation using `nse2_viz`, you can use the `viz.json` file, which provides visualization data for the nodes and contacts.

NSE2 provides a simple web-based visualization frontend to graphically represent the current state of the simulation. It displays a log file, the nodes, their (configured) positions, and the contacts between them. Optionally, it can also put a background image behind the nodes to provide a better context for the simulation, e.g., a map of the area where the nodes are located. The list of nodes also provides quick access to a terminal on each nodes.

Opening a terminal requires X11 forwarding (set `DISPLAY` environment variable) and an installed `xterm` on the host system running the manager.

The visualization frontend is started by running the `nse2_viz` command, which reads the `viz.json` file and starts a web server to serve the visualization.

```
{
  "title": "Simple NSE2 Example",
  "description": "A very simple three node dynamic network scenario.",
  "background": "extras/background.jpg",
  "links": "tmp/compose.netmap",
  "nodes": [
    {
      "name": "n1",
      "type": "computer",
      "x": 657,
      "y": 634,
      "color": "SkyBlue"
    },
    {
      "name": "n2",
      "type": "computer",
      "x": 693,
      "y": 384,
      "color": "SkyBlue"
    },
    {
      "name": "n3",
      "type": "settings_input_antenna",
      "x": 570,
      "y": 292,
      "color": "SkyBlue"
    }
  ]
}
```

A detailed description of the visualization file format can be found in Section 5.4.

To get a proper visualization of the active links, `nse2_contacts` must be started with the `-m` option to periodically generate a mapping file with all active links.

The visualization frontend is very basic but can easily be extended to provide scenario and simulation specific information.

Chapter 4. Creating a New Simulation Scenario

There are several ways to create a new simulation scenario in NSE2, depending on your preferences and requirements. This section covers the three main methods: using the `netedit` tool, converting CSV/JSON files, and manually creating a scenario file. Depending on whether you prefer a graphical interface, command-line tools, or manual editing, you can choose the method that best suits your workflow.

4.1. Manually Creating a Scenario

Creating a new simulation scenario from scratch is a straight forward process, but it requires a good understanding of the Docker Compose file format and the specific requirements for network simulation. The following sections will guide you through the steps to manually create a scenario using Docker Compose.

For a concrete example, you can refer to the simple scenario in the `scenarios/simple` directory of the NSE2 repository, which contains a basic setup with three nodes connected in a line. It is described in detail in Chapter 3, but you can also use it as a starting point for your own scenarios.

4.1.1. Create a Docker Compose File

Every new scenario starts with the creation of a Docker Compose file, which defines the network topology and the nodes in the simulation. The Compose file is written in YAML format and specifies the services (nodes), networks, and volumes that make up the simulation environment. It can be based upon an existing compose file, e.g., from a CI template or project repository, or you can start from scratch. The details about the Compose file format can be found in Section 5.1.

4.1.2. Optionally: Create a Contact Plan

The contact plan defines the communication links between the nodes and the link properties such as bandwidth, delay, loss, and jitter. It is written in a specific format that is parsed by the `nse2_contacts` tool during the simulation. The contact plan file is typically named `contacts.ccp` and is placed in the same directory as the Compose file. The details about the contact plan format can be found in Section 5.2. Of course, if no traffic shaping and no dynamic links are needed, the contact plan can be empty or not used at all.

4.1.3. Optionally: Create an Actions File

The actions file defines the actions to be performed at specific points in time during the simulation, such as starting processes, sending messages, or logging information. It is written in a simple text format that is interpreted as a shell script and can include commands to be executed on the nodes. The actions file is typically named `actions.txt` and is placed in the same directory as the Compose file. The details about the actions file format can be found in Section 5.3. If no actions are needed, the actions file can be empty or not used at all.

4.1.4. Optionally: Create a Visualization File

The visualization file is a JSON file that defines a scenario description, the nodes with their visualization properties, and where live link information can be found. It is used to visualize the network topology and the active links between the nodes. The visualization file is typically named `viz.json` and is placed in the same directory as the Compose file. The details about the visualization file format can be found in Section 5.4. If no visualization is needed, the visualization file can be empty or not used at all.

4.2. Using `netedit` to Create a Scenario (Work in Progress)

To graphically create a simulation scenario, you can use the `netedit` tool. This tool provides a user-friendly interface for designing network topologies and configuring nodes and links. It is part of the PONS-dtn project, an open source discrete event simulator for delay-tolerant networks (DTNs), and allows one to export topology with link properties in GraphML format.

To use `netedit`, you must first install the `pons-dtn` package. You can do this by running the following command in your terminal:

```
pip install pons-dtn
```

You then can start `netedit` by running:

```
netedit
```

If you want to align positions of nodes with a background image you can point the `BG_IMG` environment variable to the image file you want to use as a background. For example:

```
BG_IMG=/path/to/your/background.png netedit
```

4.2.1. Using `netedit`

The UI provides a graphical representation of the network topology, allowing you to easily add, remove, and configure nodes and links.

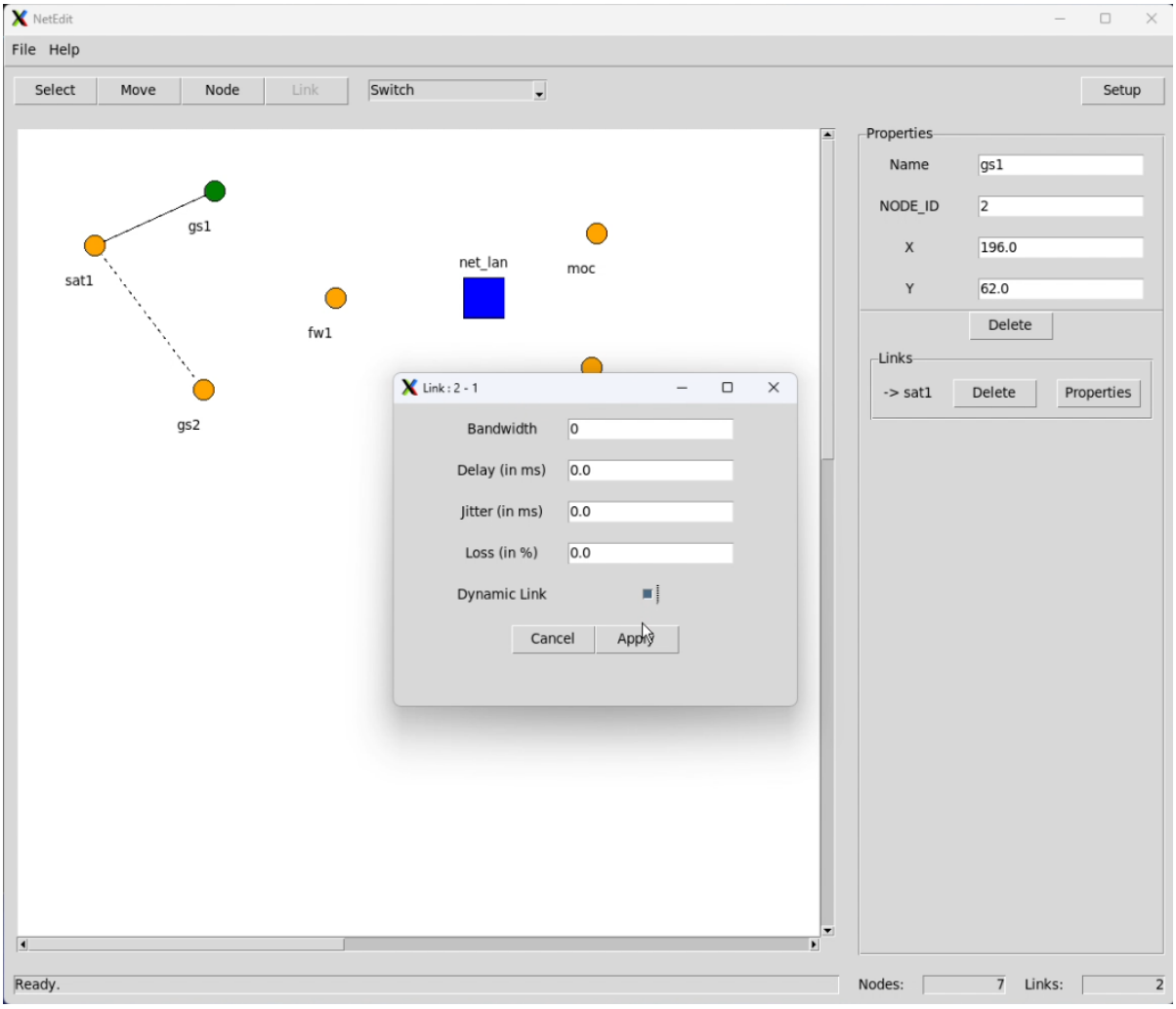

Figure 2. Netedit in action, editing the link properties of a connection between node 2 and 1.

There are 4 modes of operation as indicated by the buttons in the top left corner of the UI:

- **Select**: Select nodes to edit their properties.
- **Move**: To change the position of a node by dragging.
- **Node**: To add a new node to the topology, this can be either a *Switch* or a *Host*.
- **Link**: To create a new link between two nodes by dragging a line between them.

The currently selected node is always highlighted in green in the topology canvas.

Hosts can be either linked to another host via a 1:1 connection or to a switch, which can have multiple hosts connected to it. Switches are used to create a network segment where multiple hosts can communicate with each other.

Switch nodes should always have their names prefixed with "net_" so the automatic converters can treat them properly and it makes it easier to identify them later on.

On the right side of the UI, you can see the properties of the currently selected node and its links. You can edit the properties of the node, such as its name, type, and position, or the properties of the link, such as its bandwidth, delay, loss, and jitter.

The `NODE_ID` number must be unique in the whole topology!

In the link properties dialog, you can also set the link to be a dynamic link by checking the "Dynamic Link" checkbox. This will render the link as a dashed line, indicating that this is a non-permanent link.

Some advanced NSE2 features such as multiple parallel links between the same pair of nodes, or using specific network interfaces for the links, are not supported by `netedit` yet. You can still use the `compose.yml` file to define these features manually.

Via the *File* menu, you can save the current topology to a GraphML file, which can then be converted to a Docker Compose file.

4.2.2. Converting GraphML to Docker Compose

TODO

4.2.3. Running the Scenario

Once you have a proper Docker Compose file, you can run the scenario using the `nse2_topo` command. This command will start the Docker containers defined in the Compose file and set up the network topology as specified.

Of course, you can also add a contact plan, actions file, and visualization file as described in Section 4.1 to enhance the simulation experience.

4.3. Using `csvconvert` to Create a Scenario (Work in Progress)

TODO

Chapter 5. File Formats

5.1. *Topology* File Format (`compose.yml`)

The basic scenario and network topology is defined as a regular docker compose file.

You can start from scratch of use an existing compose file as a template. The compose file is written in YAML format and defines the services, networks, and volumes that make up the simulation environment.

```
name: simple test scenario
services:
  n1:
    container_name: n1
    hostname: n1
    cap_add:
    - NET_ADMIN
    privileged: 'true'
    volumes:
    - ./compose.yml:/compose.yml:ro
    environment:
    - NODE_ID=1
    - TYPE=Host
    networks:
      n1_n2:
        ipv4_address: 172.33.0.2
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
  n2:
    container_name: n2
    hostname: n2
    cap_add:
    - NET_ADMIN
    privileged: 'true'
    volumes:
    - ./compose.yml:/compose.yml:ro
    environment:
    - NODE_ID=2
    - TYPE=Host
    networks:
      n1_n2:
        ipv4_address: 172.33.0.3
      n2_n3:
        ipv4_address: 172.33.1.2
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
  n3:
    container_name: n3
    hostname: n3
```

```
    cap_add:
    - NET_ADMIN
    privileged: 'true'
    volumes:
    - ./compose.yml:/compose.yml:ro
    environment:
    - NODE_ID=3
    - TYPE=Host
    networks:
      n2_n3:
        ipv4_address: 172.33.1.3
    entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'
    image: alpine
networks:
  n1_n2:
    driver: bridge
    ipam:
      config:
      - subnet: 172.33.0.0/24
    driver_opts:
      com.docker.network.container_iface_prefix: n1_n2_
  n2_n3:
    driver: bridge
    ipam:
      config:
      - subnet: 172.33.1.0/24
    driver_opts:
      com.docker.network.container_iface_prefix: n2_n3_
```

5.1.1. Setting up the Nodes

Each node is represented by a service in the compose file. In addition, one should explicitly set the `container_name` and `hostname` for each node to ensure that the names are consistent across different runs of the simulation.

Furthermore, for the network emulation to work correctly, the `NET_ADMIN` capability must be added to the container. This is done by setting the `cap_add` option in the compose file. And the container should run in `privileged` mode to allow full access to the network stack. Also, the ip address of the node for a specific network can be set using the `networks` option in the compose file.

You set the container `image` or alternatively the `build` option to specify the Dockerfile to build the image for the node.

The container images used for the nodes must have `iproute2` installed so that `tc qdisc` can be used to configure the network emulation for the link effects. Also, for very large delays, e.g., several minutes like to mars, a version of `iproute2` from mid 2024 or later is required.

There are a few optional things to configure for each node:

- Sharing the compose and contact plan files `read-only` with the node container (`./compose.yml:/compose.yml:ro`, `./contacts.ccp:/contacts.ccp:ro`), so that the node can parse the topology if needed, for routing purposes.
- Set the `environment` variables `NODE_ID` to a unique number, e.g., to use for IPN auto configuration in DTN networks. This is useful for the node to identify itself in the simulation.

Of course, you can also add custom entry points or any other environment variables that are needed for the node to run, such as configuration files or other parameters specific to the service.

Sometimes it is useful to wrap your `entrypoint` commands should be wrapped in a `bash -c` command to ensure that the commands are executed in a shell environment. This is especially useful if you want to use shell features like variable expansion or command substitution. An example of this would be: `entrypoint: 'sh -c "apk add iproute2 bash && tail -f /dev/null"'`

5.1.2. Setting up the Networks

In general, it is advised to use a separate network for each pair of nodes. Thus, the network name should be unique for each pair of nodes, e.g., `n1n2` for the network between node 1 and node 2.

Network names must be unique across the entire compose file, so that they can be used to connect the nodes to the correct network. And, similar, to network interface names, network names must not contain any spaces or special characters, except for `-` and `_`, and best to keep them below 15 characters in length.

If no contact plan is used for a specific network segment, multiple nodes can be connected to the same network, e.g., `SWITCH1` for node 1 and node 2 and node 3. This is useful for scenarios where multiple nodes are connected to the same network segment, such as a LAN or a Wi-Fi network.

By default the `driver` should be set to `bridge`, which is the default Docker network driver. This allows the nodes to communicate with each other over the network.

With the default `bridge` driver, the network is not isolated from the host network, so that the nodes can access the host network (including the Internet) and vice versa. This can lead to unexpected behavior if the nodes are not properly configured to handle this. If you want to isolate the network from the host network, you can use the `macvlan` driver instead, which creates a virtual network interface for each node that is isolated from the host network.

Additionally, it can be helpful to set the `ipam` option to specify the IP address range for the network, e.g., `ipam: {config: [{subnet: "172.33.0.0/24"}]}`.

Finally, if you expect multiple links between the same pair of nodes, you can use specific network interfaces for the link (`com.docker.network.container_iface_prefix`). This prefix is used for the actual network interface name in the node container. Contact plans can then refer to these interfaces by their name, e.g., `dev:eth0` or `dev:n1_n2_0` for the second interface, and so on, instead of using the node name as second node.

Network names in docker are global. Thus, running two scenarios with the same network names in parallel will lead to conflicts!

5.2. *Core Contact Plan* File Format (`contacts.ccp`)

The format of the contact plan file is `.ccp`, which stands for *Core Contact Plan* and is taken from the Core Contact Manager, a fluctuating connectivity simulator for the core network emulator. This file defines the communication links between nodes, including properties such as bandwidth, delay, loss, and jitter. The contact plan is essential for simulating realistic network conditions in NSE2.

Links between nodes can be fixed or fluctuating, and the plan can either be configured to have symmetric (one line per node pair) or asymmetric links (two lines per node pair - one per direction).

Each dynamic contact is defined by a time window, which specifies when the contact is active. Other tools can be used to simulate actual node movement and calculate visibility for the contact plan from this. The contact plan file is structured in a way that allows for easy parsing and integration with NSE2.

```
# each entry starts with "a contact" or "a fixed"
# a fixed <node id 1 OR name> <node id 2 OR name OR dev:<interfacename> > <bandwidth>
[loss%] [delay] [jitter]
# a contact <begin timestamp in seconds> <end timestamp in seconds> <node id 1 OR
name> <node id 2 OR name OR dev:<interfacename> > <bandwidth> [loss%] [delay] [jitter]

s loop 1
a fixed n1 n2 100000000 0.0 0.1 0.0
a fixed n2 n1 100000000 0.0 0.1 0.0
a contact 20 40 n2 n3 2000000 0.0 0.01 0.0
a contact 20 40 n3 n2 2000000 0.0 0.01 0.0
```

Optionally, variables such as `loop` can be `s` et to 1 to indicate that the contact plan should be repeated indefinitely if the simulation runs longer than the last timestamp. This can also be overriden by the `--loop` command line option when starting the simulation with `nse2_contacts`.

The first node must always be specified by its name, while the second node can be specified by its name or by its interface name (e.g., `dev:eth0`). This allows for flexibility in defining contacts, especially in scenarios where nodes have multiple interfaces to the same next-hop node or are part of a larger network topology.

For convenience, the bandwidth can be specified in various units, such as `kbit`, `mbit` or `gbit`.

```
a contact 20 40 n3 n2 2mbit 0.0 0.01 0.0
```

When a contact plan was designed with symmetric links in mind, it is necessary to use the `--symmetric` command line option when starting the simulation with `nse2_contacts` to automatically create the second line for each contact.

You can interact with the contact plan file using `nse2_cmd` to get information about the running scenario, current links, and other properties. Additionally, you can control the playback of the contact plan, such as pausing or resuming the simulation, skipping to the next event, etc. This is useful for debugging and testing scenarios with dynamic contacts.

5.3. *Actions* File Format (`actions.txt`)

Action files define a number of actions or commands that are executed during different points in time during the simulation. These actions can be used to control the behavior of the nodes, such as starting or stopping services, sending messages, or changing configurations.

The actions file is written in a simple text format, where each line represents an action. A simple example can be seen in the following snippet:

```
# DURATION is optional
# DURATION=300

# start a process to log the link properties every 10 seconds on the ground stations
EXEC 0 n1 "date > /tmp/started.txt"
EXEC 0 n2 "date > /tmp/started.txt"
EXEC 0 n3 "date > /tmp/started.txt"

# at timestamp 5 start to send ping (200 bytes) every 10s to gs1 from eosat
REPEAT 5 10 n1 "(date ; ping -c 1 -s 200 -W1 -w1 n2; echo "----------------------") >>
/tmp/n2.txt"

# at timestamp 10 start to send ping (200 bytes) every 10s to gs2 from eosat
REPEAT 10 10 n2 "(date ; ping -c 1 -s 200 -W1 -w1 n3; echo "----------------------")
>> /tmp/n3.txt"

# be nice and shut down long running processes
FINALLY n1 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n2 "date > /tmp/finished.txt && pkill -f ping"
FINALLY n3 "date > /tmp/finished.txt && pkill -f ping"
```

There is one important but *optional* global variable, `DURATION`, which is set to the duration of the simulation, or more precisely, the runtime of the action script in seconds. Without this the actions run until the action runner process is stopped.

There are three types of actions that can be defined in the actions file:

Table 1. Action Commands

Command	Parameters	Description
`EXEC`	`<timestamp> <node_id> <command>`	Executes a command *once* on the specified node at the given timestamp.
`REPEAT`	`<timestamp> <interval> <node_id> <command>`	*Repeats* a command on the specified node starting at the given timestamp with the specified interval.
`FINALLY`	`<node_id> <command>`	Executes a command on the specified node at the end of the simulation or when the action runner is stopped.

Action scripts are interpreted as shell scripts, thus, you can also use environment variables in your scripts. For example, you can set something like a shared `$SEND_INTERVAL` to be reused by all `REPEAT` actions.

There can be only one entry per time and node or in the case of `FINALLY` one entry per node. If there are multiple entries for the same time and node, only the last one will be executed. This is to ensure that the actions are deterministic and do not interfere with each other. If you want to execute multiple commands at the same time, you can use a shell script that contains all the commands and call it from the action file or just chain them with `&&`.

5.4. *Visualization* File Format (`viz.json`)

The visualization file format is a JSON file that defines the nodes and links in the simulation scenario. It is used to visualize the network topology and the active links between the nodes. The file contains a list of nodes, each with a name, type, position (x and y coordinates), and color. Optionally, it can also put a background image behind the nodes to provide a better context for the simulation, e.g., a map of the area where the nodes are located as shown in Figure 3.

Figure 3. Example visualization file of an earth observation scenario.

The visualization frontend is started by running the `nse2_viz` command, which reads the `viz.json` file and starts a web server to serve the visualization.

The `viz.json` file used for Figure 3 is as follows:

```
{
  "title": "Earth Observation",
  "description": "A minimal earth observation scenario.",
  "background": "extras/background.jpg",
  "links": "tmp/eo.compose.netmap",
  "nodes": [
    {
      "name": "pcc",
      "type": "computer",
      "x": 657,
      "y": 634,
      "color": "SkyBlue"
    },
    {
      "name": "mcc",
      "type": "computer",
      "x": 693,
      "y": 384,
      "color": "SkyBlue"
    },
    {
      "name": "gs1",
      "type": "settings_input_antenna",
      "x": 570,
      "y": 292,
      "color": "SkyBlue"
    },
    {
      "name": "gs2",
      "type": "settings_input_antenna",
      "x": 410,
      "y": 544,
      "color": "SkyBlue"
    },
    {
      "name": "eosat",
      "type": "satellite_alt",
      "x": 200,
      "y": 400,
      "color": "SkyBlue"
    }
  ]
}
```

The most important top level keys in the JSON file are:

Table 2. Important visualization JSON keys

Key	Description
`title`	An optional title for the visualization.
`description`	An optional description of the visualization.
`background`	An optional background image to be displayed behind the nodes.
`links`	A file containing the current state of links.
`nodes`	A list of nodes in the simulation, each with a unique name, type, position, and color.

The `links` key is used to specify the file that contains the current state of links in the simulation. This file is generated by the `nse2_contacts` tool, which collects the active links between the nodes during the simulation. The file just contains a list of links, each pair on a separate line and a "-" or "." inidicating whether its a fixed or dynamic link, e.g., `n1 - n2` or `n1 . n2`.

To get a proper visualization of the active links, `nse2_contacts` must be started with the `-m` option to periodically generate a mapping file with all active links.

5.5. Optional: Node File Format (`node.yml`)

TODO

See CCSDS DTN Reference Scenarios Orange Book Draft for details on the node file format.

5.6. Optional: Contacts File Format (`contacts.csv`)

TODO

See CCSDS DTN Reference Scenarios Orange Book Draft for details on the node file format.